Forward, Backward, and the Partition

From the Delannoy King to Feynman and Gibbs

A Companion Piece on Path Ensembles, Forward-Backward, and the Partition Function

Lee McCulloch-James
Department of Natural & Unnatural Sciences
Dwight School London

May 23, 2026

Abstract

This is the second half of a pair. The first piece, *The King's Diagonal* [1], counted lattice paths on the chessboard. Here we ask a more physical question: if each path carries a weight, what does the *ensemble* of paths look like? We arrive at three ideas that anchor the whole of computational physics and modern machine learning: the **partition function** as the normalising sum over histories, the **Gibbs–Feynman weight** $e^{-\beta L}$ as the natural penalty on long paths, and the **forward–backward decomposition** that turns an apparently intractable weighted enumeration into an $O(n^2)$ computation. The same decomposition animates hidden Markov models [8, 9], belief propagation [12], and the backpropagation algorithm in neural networks [15]. An accompanying interactive visualiser [2] renders the ensemble in real time under a slider over β. No physics is assumed; only the Delannoy recurrence from the previous piece.

1 Recall: The King and His Three Parents

We left the king at a corner of an $n \times n$ grid [1]. From the origin $(0, 0)$ he walks to (n, n) using the three steps East, North, and North-East, never retreating.
The number of distinct paths is the Delannoy number $D(n, n)$, satisfying the three-parent recurrence [3]

$$D(i, j) = D(i-1, j) + D(i, j-1) + D(i-1, j-1) \tag{1}$$

with $D(0, 0) = 1$ and $D(i, j) = 0$ outside the lattice. The recurrence generates a table whose first eight rows and columns are:

$$D = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 3 & 5 & 7 & 9 & 11 & 13 & 15 \\ 1 & 5 & 13 & 25 & 41 & 61 & 85 & 113 \\ 1 & 7 & 25 & 63 & 129 & 231 & 377 & 575 \\ 1 & 9 & 41 & 129 & 321 & 681 & 1289 & 2241 \\ 1 & 11 & 61 & 231 & 681 & 1683 & 3653 & 7183 \\ 1 & 13 & 85 & 377 & 1289 & 3653 & 8989 & 19825 \\ 1 & 15 & 113 & 575 & 2241 & 7183 & 19825 & \mathbf{48639} \end{pmatrix}$$

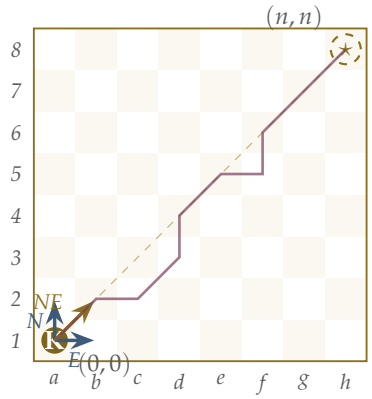

Figure 1: The king walks from a1 to h8 using only East, North, and North-East moves. Three legal first moves are shown in colour; one example completed path is drawn faintly across the board. The dashed line marks the Euclidean geodesic.

Each interior entry is the sum of its three immediate neighbours to the West, South, and South-West. The boundary 1s come from the unique single-direction walks along each axis. The bottom-right corner gives the chessboard answer $D(7,7) = 48{,}639$: the king has 48,639 distinct ways to walk from a1 to h8.

This counted every path equally. But the three steps are not metrically equivalent — E and N each cover a Euclidean length of 1, while NE covers $\sqrt{2}$. The king walking pure-orthogonal traverses $2n$ units of Euclidean length to reach (n, n); the king walking pure-diagonal traverses only $n\sqrt{2}$. *Equal counting is not equal weight.*

We now ask: what if we weight each path by its Euclidean length?

2 The Boltzmann–Feynman Weight

Assign to a path γ with total Euclidean length $L(\gamma)$ the weight

$$w(\gamma) = e^{-\beta L(\gamma)}. \tag{2}$$

The parameter β is positive when long paths are penalised, negative when they are rewarded, and zero when all paths are equal. In thermal physics $\beta = 1/(k_B T)$ is the inverse temperature [4]. In Feynman's formulation of quantum mechanics the analogous weight is $e^{iS/\hbar}$, oscillating rather than decaying [5]; the Wick rotation $t \to -i\tau$ turns one into the other.

A single step contributes its length to $L(\gamma)$. So the weight of a path *factorises across steps*:

$$w(\gamma) = \prod_{\text{steps } s \in \gamma} e^{-\beta \ell(s)} \tag{3}$$

where $\ell(s) \in \{1, 1, \sqrt{2}\}$ for an E, N, or NE step. This factorisation is the algebraic foundation of everything that follows. The moment weights multiply along a path, all the machinery of recurrences and matrix products becomes available.

The Boltzmann weight $e^{-\beta L}$ converts additive *accounting of path length into* multiplicative *accounting of path weight. Each step contributes one factor.*

---

3 The Partition Function

Sum the weights over all paths from origin to destination:

$$Z_n(\beta) = \sum_{\gamma:(0,0)\to(n,n)} e^{-\beta L(\gamma)}. \tag{4}$$

This is the **partition function** [4]. Three of its properties make it the central object of the theory.

- At $\beta = 0$, every weight is 1 and $Z_n(0) = D(n,n)$: the Delannoy count. The partition function is the natural generalisation of "number of paths" to weighted ensembles.
- It is the *normalising denominator* for probabilities. The probability that the king follows path γ is $\Pr(\gamma) = w(\gamma)/Z_n(\beta)$. This is the Gibbs distribution.
- It is a *generating function* for averages. The expected length, for instance, is

$$\langle L \rangle = -\frac{\partial \log Z_n}{\partial \beta}, \tag{5}$$

  which one derives by differentiating $Z_n = \sum e^{-\beta L}$ under the sum and dividing by Z_n. The energy of a thermal system, the magnetisation of a spin lattice, and the expected loss of a probabilistic classifier all arise this way.

Aside. *The word "partition" came from Boltzmann's 19th-century probabilistic account of gases: Z partitions the unit total probability among possible microstates. Statistical mechanics texts often write Q or Ξ; physics papers prefer Z from the German* Zustandssumme*, "sum over states." The latter usage is now universal.*

4 The Apparent Computational Catastrophe

A direct attempt to compute $Z_n(\beta)$ enumerates paths. For $n = 7$ (the chessboard) there are 48,639 of them; for $n = 20$ there are roughly 2×10^{14}. The enumeration is exponentially impossible before we even reach a respectable lattice size.

Worse: many quantities we want such as the *visit probability* $P(i,j)$, the chance that a sampled path passes through cell (i,j) seemingly require us to identify, for each cell, which paths visit it. This appears to be a doubly-exponential mess.

The salvation is that every path that visits (i,j) can be split into two *independent* halves: a sub-path from $(0,0)$ to (i,j), and a sub-path from (i,j) to (n,n).

5 Forward and Backward Sums

Define two tables, both of size $(n+1) \times (n+1)$.

Forward weighted sums

Let $F(i,j)$ be the total weight of all paths from $(0,0)$ to (i,j):

$$F(i,j) = \sum_{\gamma:(0,0)\to(i,j)} e^{-\beta L(\gamma)}. \tag{6}$$

A path arriving at (i,j) took its last step from one of three predecessors. The Boltzmann weight factorises, so:

$$F(i,j) = e^{-\beta}\, F(i-1,j) + e^{-\beta}\, F(i,j-1) + e^{-\beta\sqrt{2}}\, F(i-1,j-1), \tag{7}$$

with base case $F(0,0) = 1$.

This is the Delannoy recurrence (1) *with each parent contribution multiplied by the weight of the step from that parent*. At $\beta = 0$ the weights are all 1 and $F(i,j) = D(i,j)$.

Backward weighted sums

Let $B(i,j)$ be the total weight of paths from (i,j) to (n,n):

$$B(i,j) = \sum_{\gamma:(i,j)\to(n,n)} e^{-\beta L(\gamma)}. \tag{8}$$

By the same logic applied to the first step of each remaining sub-path:

$$B(i,j) = e^{-\beta}\, B(i+1,j) + e^{-\beta}\, B(i,j+1) + e^{-\beta\sqrt{2}}\, B(i+1,j+1), \tag{9}$$

with base case $B(n,n) = 1$.

The two recurrences are mirror images: forward fills the table from origin outward toward the destination; backward fills it from destination inward.

The join

Observe that $F(n,n) = Z_n(\beta) = B(0,0)$. Both sums, evaluated at their respective "opposite" corners, give the partition function. Either recurrence on its own suffices to compute Z. But for visit probabilities we need both.

6 The Forward-Backward Product

Consider all paths passing through a fixed cell (i,j). Each such path splits at (i,j) into a head and a tail, with no constraint linking them except their meeting point. The weight of the whole path is the weight of the head times the weight of the tail. Summing over all heads and all tails:

$$\boxed{\sum_{\substack{\gamma:(0,0)\to(n,n)\\ \gamma\ni(i,j)}} e^{-\beta L(\gamma)} \;=\; F(i,j)\cdot B(i,j)} \tag{10}$$

The visit probability is then the ratio of this to the total partition function:

$$P(i,j) = \frac{F(i,j)\cdot B(i,j)}{Z_n(\beta)}. \tag{11}$$

The forward-backward identity is just the combinatorial product rule applied to weighted sums: the weight of all paths through a cell equals the weight of arrivals at the cell, times the weight of departures from the cell. *The Boltzmann factorisation makes this exact.*

---

7 The $O(n^2)$ Cost

Each entry of F takes constant time given its three predecessors. The table has $(n+1)^2$ cells. So filling F costs $O(n^2)$ arithmetic operations. The same for B. The product map $F \cdot B$ and the normalisation are another $O(n^2)$. So all visit probabilities, for the entire grid, are computed in $O(n^2)$ time. This is a textbook instance of dynamic programming [7].

n	paths $D(n,n)$	naïve enumeration	forward-backward	speedup
5	1 683	1 683	36	$\sim 50\times$
7	48 639	48 639	64	$\sim 750\times$
10	8 097 453	$\sim 8 \times 10^6$	121	$\sim 7 \times 10^4\times$
20	$\sim 2 \times 10^{14}$	infeasible	441	$\sim 5 \times 10^{11}\times$
50	$\sim 10^{37}$	impossible	2 601	—

The Delannoy count grows like $(3+2\sqrt{2})^n/\sqrt{n}$ — exponential in n. The forward-backward cost grows like n^2. The ratio is a savings of order $5.83^n/n^2$, which becomes astronomical by $n = 20$. *This is the algorithmic miracle that makes the visualisation possible.*

8 Pseudocode

Algorithm 1 Forward-backward on the Delannoy lattice

$w_E \leftarrow e^{-\beta}$, $w_N \leftarrow e^{-\beta}$, $w_D \leftarrow e^{-\beta\sqrt{2}}$
$F(0,0) \leftarrow 1$
for $i = 0$ to n **do**
    for $j = 0$ to n **do**
        if $(i,j) \neq (0,0)$ **then**
            $F(i,j) \leftarrow w_E F(i{-}1,j) + w_N F(i,j{-}1) + w_D F(i{-}1,j{-}1)$
        end if
    end for
end for
$B(n,n) \leftarrow 1$
for $i = n$ down to 0 **do**
    for $j = n$ down to 0 **do**
        if $(i,j) \neq (n,n)$ **then**
            $B(i,j) \leftarrow w_E B(i{+}1,j) + w_N B(i,j{+}1) + w_D B(i{+}1,j{+}1)$
        end if
    end for
end for
$Z \leftarrow F(n,n)$
for each cell (i,j) **do**
    $P(i,j) \leftarrow F(i,j) \cdot B(i,j)/Z$
end for

In Python, this is about thirty lines. The artefact accompanying this piece [2] runs it in JavaScript, in the browser, every time the user moves the inverse-temperature slider.

9 Two Slightly Different Quantities, One Algorithm

The same forward and backward tables furnish many other quantities.

Expected number of diagonal steps

The probability that the step from $(i-1, j-1)$ to (i, j) is taken by *some* path in the ensemble equals

$$\Pr(\text{NE step into } (i,j)) = \frac{F(i-1,j-1) \cdot w_D \cdot B(i,j)}{Z}. \tag{12}$$

Summing over all (i, j) gives the expected number of diagonal steps in a random path. The corresponding sums over E-edges and N-edges give the expected numbers of orthogonal moves. The expected length is then

$$\langle L \rangle = \langle n_E \rangle + \langle n_N \rangle + \sqrt{2}\langle n_D \rangle. \tag{13}$$

Marginal probability of a specific step

For any single edge in the lattice, the same formula gives the probability that a sampled path uses that edge. This is the marginal of the Gibbs distribution over a single bond. Plotting all edge marginals gives a flow diagram of where probability mass moves.

Sampling a path

To draw a sample path from the Gibbs distribution: starting at the origin, look at the three outgoing edges from the current cell and assign each its forward-conditional probability,

$$\Pr(\text{step } s \text{ from } (i,j)) \propto w_s \cdot B(\text{next cell after step } s), \tag{14}$$

then pick one randomly with these probabilities and repeat. This is exact sampling — not Monte Carlo — and it runs in $O(n)$ per sample because each step is independent given the backward table.

10 Why This Is the Same as Backpropagation

The reader familiar with neural networks will recognise the pattern [15, 16]. Replace "cell on a lattice" with "neuron at a layer," "path weight" with "activation product," and "partition function" with "output loss".

As such the algorithm is identical in structure.

Delannoy path ensemble	**Neural network**
Cell (i, j) on a grid	Neuron j at layer i
Step from cell to neighbour	Connection from neuron to neuron
Step weight $w_s = e^{-\beta \ell(s)}$	Edge weight W_{ij}
Forward sum $F(i, j)$	Activation forward pass $a_j^{(i)}$
Backward sum $B(i, j)$	Gradient backward pass $\delta_j^{(i)}$
Partition function $Z = F(n, n)$	Loss $\mathcal{L}$ at output layer
Visit probability $F \cdot B / Z$	Edge contribution to loss / sensitivity

In both cases:

- The forward pass propagates a multiplicative quantity from input to output, summing over the ways the input can become the output.
- The backward pass propagates a quantity from output back to input, summing over the ways the output depends on the input.
- Their product, at any intermediate node, tells you the *importance* of that node to the overall computation.

In machine learning the backward pass is called "backpropagation of errors" because it transports the gradient of the loss. The Delannoy analogue transports the partition contribution of the future. They are the same operation under different names. The credit for recognising this in its full generality is usually given to Judea Pearl, whose belief propagation algorithm of 1982 unified message-passing across statistical inference, statistical mechanics, and probabilistic AI [12, 13].

Aside. *The Baum–Welch algorithm for hidden Markov models [8], the Viterbi algorithm for decoding [10], the inside-outside algorithm for probabilistic context-free grammars [11], and the sum-product algorithm for graphical models [14] are all special cases of the same forward-backward decomposition. Each field discovered it independently and named it after its inventor.*

11 The Feynman Bridge

We return to physics for the closing observation. Feynman's path integral [5, 6] defines the quantum amplitude for a particle to go from x_i at time t_i to x_f at time t_f as the sum over all paths γ connecting them, weighted by $e^{iS(\gamma)/\hbar}$:

$$K(x_f, t_f; x_i, t_i) = \int_{\gamma:(x_i,t_i)\to(x_f,t_f)} \mathcal{D}\gamma \; e^{iS[\gamma]/\hbar}. \tag{15}$$

The fundamental property of K, which is the property that makes the whole formalism work, is its *composition law*:

$$K(x_f, t_f; x_i, t_i) = \int dx' \; K(x_f, t_f; x', t') \; K(x', t'; x_i, t_i). \tag{16}$$

This says: the amplitude to go from start to finish equals the sum, over all possible intermediate states (x', t'), of the amplitude to reach the intermediate state times the amplitude to continue from there. It is exactly equation (10) in continuous form.

Feynman's composition law is the continuous limit of our forward-backward identity. *The Delannoy lattice is a discrete toy model of the path integral. Forward × Backward / Partition Function is the discrete propagator.*

When physicists say "the propagator factorises through intermediate states," or "the wavefunction is the sum over all histories times the amplitude of the future," they are saying what we have just done with F and B. The same algebraic structure with weights that multiply along paths, sums that factorise across split-points that supports the path integral, the partition function, the Markov chain, the belief propagation algorithm, and the training of neural networks.

12 Closing Thought

The king's diagonal began as a counting puzzle: *how many ways can a king cross the chessboard?* We answered 48,639, and built the recurrence that generates the answer.

But the same recurrence, decorated with a single Boltzmann weight, becomes the algorithm at the heart of statistical mechanics and machine learning. The partition function $Z_n(\beta)$ is the natural object; the forward-backward decomposition is the natural way to compute its derivatives; and the multiplicative factorisation of path weights is the algebraic identity that makes everything tractable.

What the king teaches us, in the end, is that whenever a problem admits a notion of *path*, a notion of *weight along a path*, and a notion of *splitting at an intermediate point*, the algorithm that computes its ensemble averages is the same. We met it on a chessboard and we will meet it again in every other place that paths are weighted and summed.

Exercise. *Verify by hand on the* 2×2 *grid that* $F(1,1) \cdot B(1,1)/Z_2(0) = 9/13$*, agreeing with the direct count: of the* 13 *paths to* $(2,2)$*, nine pass through the central cell* $(1,1)$*.*

Exercise. *At* $\beta = 0$*, show that* $F(i,j) = D(i,j)$ *(the Delannoy number from the previous piece) and that* $B(i,j) = D(n-i, n-j)$*. Conclude that* $D(n,n) = \sum_{(i,j)} D(i,j)D(n-i,n-j)/D(n,n)$*, which is a striking self-similarity of the Delannoy table.*

Exercise. *Differentiate* $\log Z_n(\beta)$ *with respect to* β *and confirm that the result is* $-\langle L \rangle$*.* Hint: differentiate inside the sum. *This is the simplest example of a* thermodynamic identity*: an average computed by differentiating a logarithm of a sum.*

Exercise. *Suppose you only had the forward table* F*, not the backward table* B*. Could you still compute* $Z_n(\beta)$*? Could you compute* $\langle L \rangle$*? Could you compute the visit probabilities* $P(i,j)$*? Explain which questions remain answerable and which become intractable.*

References

[1] McCulloch-James, L. *The King's Diagonal: Counting Lattice Paths Across the Chessboard.* McMurmerings Pedagogical Series, 2026. `https://leelemacjames.github.io/Dellanoy_Intro.pdf`

[2] McCulloch-James, L. *The Weighted King — a Feynman ensemble on the Delannoy lattice.* Interactive HTML artefact, McMurmerings, 2026. `https://leelemacjames.github.io/feynman_delannoy.html`

[3] Sulanke, R. A. *Objects counted by the central Delannoy numbers.* Journal of Integer Sequences, 6, 2003, Article 03.1.5.

[4] Gibbs, J. W. *Elementary Principles in Statistical Mechanics.* Yale University Press, 1902.

[5] Feynman, R. P. *Space-time approach to non-relativistic quantum mechanics.* Reviews of Modern Physics, 20(2), 1948, pp. 367–387.

[6] Feynman, R. P. and Hibbs, A. R. *Quantum Mechanics and Path Integrals.* McGraw-Hill, 1965.

[7] Bellman, R. *Dynamic Programming.* Princeton University Press, 1957.

[8] Baum, L. E. and Welch, L. R. *An inequality with applications to statistical estimation for probabilistic functions of Markov processes.* Inequalities, 3, 1972, pp. 1–8.

[9] Rabiner, L. R. *A tutorial on hidden Markov models and selected applications in speech recognition.* Proceedings of the IEEE, 77(2), 1989, pp. 257–286.

[10] Viterbi, A. J. *Error bounds for convolutional codes and an asymptotically optimum decoding algorithm.* IEEE Transactions on Information Theory, 13(2), 1967, pp. 260–269.

[11] Lari, K. and Young, S. J. *The estimation of stochastic context-free grammars using the inside-outside algorithm.* Computer Speech and Language, 4(1), 1990, pp. 35–56.

[12] Pearl, J. *Reverend Bayes on inference engines: A distributed hierarchical approach.* Proceedings of the Second AAAI Conference on Artificial Intelligence, 1982, pp. 133–136.

[13] Pearl, J. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference.* Morgan Kaufmann, 1988.

[14] Kschischang, F. R., Frey, B. J., and Loeliger, H.-A. *Factor graphs and the sum-product algorithm.* IEEE Transactions on Information Theory, 47(2), 2001, pp. 498–519.

[15] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. *Learning representations by back-propagating errors.* Nature, 323, 1986, pp. 533–536.

[16] LeCun, Y. *A theoretical framework for back-propagation.* Proceedings of the 1988 Connectionist Models Summer School, Morgan Kaufmann, 1988, pp. 21–28.

Constructed by Silicon · Anthropic Claude — Architect Carbon · LeeMcCullochJames